

CODER PROMPT — BUILD DETERMINISTIC PARITY VERIFICATION HARNESS: NO MORE “WAIT FOR LIVE SESSION” DEPENDENCY

We have core production verification of the canonical alert pipeline, but final/full field parity is still pending because live-market verification requires a real Kite session and market-hours signal.

This is not acceptable as an ongoing workflow.

We need a deterministic verification harness so that app ↔ DB ↔ paper trading ↔ Telegram payload parity can be verified safely outside live market hours, without fake production data, without real orders, and without sending fake trade-looking Telegram messages.

Current accepted status:

`SIGNAL_DATA_NOTIFICATION_PARITY_CORE_PROD_VERIFIED_OPERATIONAL_ITEMS_PENDING`

Known verified:

- Canonical trade lifecycle pipeline is live.
- `swingAlerts.ts` uses canonical validator/formatter/dedup.
- `fnoSignalAlerts.ts` uses canonical validator/formatter/dedup.
- `paperTradingFO.ts` exit alert fires after close transaction commit.
- `notification_delivery_log` is active in production.
- `TESTSTK` is blocked.
- Dev/test events are blocked.
- Duplicate lifecycle delivery violations are zero.
- Broker execution remains disabled.
- No real orders are placed.

Remaining issue:

Final field parity across:

- F&O Page
- Swing Queue
- Paper Trading
- Telegram message
- DB rows
- `notification_delivery_log`

still requires live-market proof.

We need to solve this by adding a production-safe parity verification harness.

===== OBJECTIVE
=====

Create a deterministic, production-safe parity verification system that can verify the same canonical trade event across all surfaces without waiting for a live signal.

This must not fake production signals.

This must not send real trade-looking Telegram alerts.

This must not mutate real trading state except explicitly marked verification/audit rows if required.

This must not enable broker execution.

This must not place real orders.

The harness must prove that:

1. The same canonical event payload is used by UI, DB, paper trading, and Telegram formatter.
2. Field values match across surfaces.
3. Test/sample/dev events cannot reach production Telegram.
4. Duplicate lifecycle events are blocked.
5. Suppressed/info-only/stale events do not produce trade alerts.
6. Exit events are sent only once and only after close commit.
7. Real production alert path remains protected.

===== DO NOT DO
=====

Do not change F&O strategy logic.

Do not change Swing scoring logic.

Do not change thresholds.

Do not enable broker execution.

Do not place real orders.

Do not send fake trade-looking alerts to the real Telegram trade channel.

Do not run heavy candle backfill.

Do not delete DB rows.

Do not hide source warnings.

Do not mark simulated data as live.

Do not claim live-market verification if the test uses a fixture/replay.

Do not trim or rewrite `replit.md`.

===== PART A — ADD PARITY VERIFICATION
MODE =====

Add a controlled parity verification mode for owner-only diagnostics.

Suggested endpoint group:

```
/api/parity/trade-event/verify
/api/parity/trade-event/replay/:eventId
/api/parity/trade-event/latest
```

Owner-only strict auth required.

The endpoint must support:

1. `mode="dry_run"`
2. builds canonical event
3. runs validation
4. formats Telegram message
5. checks dedup key
6. compares projected UI payload
7. does not send Telegram
8. does not create paper trade
9. does not mutate real state
10. `mode="test_destination"`
11. sends only to clearly test-only Telegram destination if configured
12. message title must say `TEST PARITY VERIFICATION – NOT A TRADE`
13. must not go to real trade Telegram channel
14. `mode="replay_existing"`
15. loads existing real notification_delivery_log / paper_trade / swing_order_staging / option_signal record
16. reconstructs canonical event
17. verifies values match across DB, formatter, and UI projection
18. no Telegram send

Reject any production trade-channel send in this endpoint.

===== PART B — CREATE CANONICAL UI PROJECTION =====

Create a shared projection function:

```
projectTradeEventForUi(event: CanonicalTradeEvent)
```

This is the object that UI cards should render from or compare against.

It must include:

- eventId
- domain
- lifecycleStatus
- eventType
- signalId
- orderId
- paperTradeId
- symbol
- tradingSymbol
- exchange
- instrumentToken
- underlying
- contract/strike/CE/PE for F&O
- entry
- stopLoss
- target1
- target2
- exitPrice
- exitReason
- qty/lots
- risk
- riskPercent
- capitalRequired
- source
- sourceStatus
- sourceAsOf
- confidence
- brokerExecutionStatus
- paperTradeStatus
- environment
- createdAt
- entryTime
- exitTime
- warnings

Rules:

- Telegram formatter must read from the same canonical event.
- UI projection must read from the same canonical event.
- Paper trade snapshot comparison must read from the same canonical event.
- Do not duplicate calculations.

```
===== PART C — FIELD PARITY COMPARATOR
=====
```

Create a comparator:

```
compareTradeEventParity({  
    canonicalEvent,  
    uiProjection,  
    telegramPreview,  
    dbSnapshot,  
    paperTradeSnapshot,  
    notificationLogSnapshot  
})
```

It must compare:

Swing fields:

- orderId
- symbol
- exchange
- instrumentToken
- entry
- stopLoss
- target1
- target2
- qty
- risk
- riskPercent
- capitalRequired
- source
- sourceAsOf
- brokerExecutionStatus
- paperTradeStatus
- lifecycleStatus

F&O fields:

- signalId
- paperTradeId
- underlying
- contract
- strike
- CE/PE
- entryPremium
- stopLoss
- target
- qty/lots
- source
- sourceAsOf
- confidence
- lifecycleStatus
- paperTradeStatus

- brokerExecutionStatus

Result shape:

```
{
  ok: boolean;
  eventId: string;
  mismatches: Array<{
    field: string;
    canonical: unknown;
    ui: unknown;
    telegram: unknown;
    db: unknown;
    paperTrade: unknown;
    notificationLog: unknown;
    severity: "P0" | "P1" | "P2";
  }>;
  blockedReasons: string[];
  warnings: string[];
}
```

Any mismatch in price, symbol, source, lifecycle status, or ID is P0.

===== PART D — REPLAY EXISTING
 PRODUCTION RECORDS =====

Use existing production records to verify now, without waiting for market hours.

Replay:

1. latest Swing notification_delivery_log records
2. latest F&O notification_delivery_log records
3. latest swing_order_staging rows
4. latest paper_trade_fo rows
5. latest paper_trade_eq rows if relevant

For each record:

- reconstruct canonical event
- generate UI projection
- generate Telegram preview
- compare DB snapshot
- compare paper trade snapshot where available
- compare notification log
- report mismatch

Do not send Telegram during replay.

Create report table:

Event	Domain	Source Record	Parity OK?	Mismatches	Action
-------	--------	---------------	------------	------------	--------

===== PART E — SAFE FIXTURE-BASED TESTS =====

Add deterministic fixtures for:

1. Valid Swing `ENTRY_READY`
2. Swing approval after entry-ready
3. Swing dry-run after entry-ready
4. Swing exit stop-loss
5. Swing exit target
6. Valid F&O `ENTRY_OPENED`
7. F&O suppressed/info-only
8. F&O exit stop-loss
9. F&O exit target
10. TESTSTK blocked
11. Yahoo source blocked
12. Stale source blocked
13. Dev environment blocked
14. Duplicate lifecycle event blocked

Fixtures must be clearly marked:

```
fixtureOnly: true
environment: "test"
sendTelegram: false
```

No fixture may use real production Telegram destination.

No fixture may create real paper trade unless inserted into a transaction that rolls back.

===== PART F — PRODUCTION-SAFE DRY RUN =====

Run production-safe dry-run verification now.

Required dry-run checks:

1. Build valid Swing canonical event using a real valid instrument.
2. Validate it.
3. Format Telegram preview.
4. Build UI projection.
5. Compare fields.
6. Do not send Telegram.
7. Do not create paper trade.
8. Build valid F&O canonical event using valid index/option contract snapshot or existing DB record.

9. Validate it.
10. Format Telegram preview.
11. Build UI projection.
12. Compare fields.
13. Do not send Telegram.
14. Do not create paper trade.
15. Build invalid `TESTSTK` event.
16. Confirm blocked at `TEST_SYMBOL_BLOCKED`.
17. Confirm no Telegram.
18. Confirm no paper trade.
19. Build stale/Yahoo event.
20. Confirm blocked.
21. Confirm no Telegram.
22. Confirm no paper trade.

===== PART G — F&O LIVE VS PROD
 MISMATCH DIAGNOSTIC WITHOUT LIVE WAIT
 =====

Even outside market hours, verify why live/prod can diverge.

Add diagnostic fields to F&O signal API and diagnostics if not already present:

- cycleId
- setupSnapshotId
- cycleStartedAt
- cycleCompletedAt
- sourceAsOf
- cacheAgeSec
- isStale
- originProcess
- environment
- warmupResultId or warmupCompletedAt
- dailyBarsStatus
- intradayBarsStatus
- optionChainStatus
- signalCount
- suppressedCount
- suppressionReasons

Then compare:

- published live `/options` backing API
- Replit dev/prod `/options` backing API
- `/api/data-health/backbone`

- `/api/fno/no-signal-gap`
- `/fno-diagnostics`

Outside live market, expected result may be “market closed / no fresh signal,” but both environments must explain state using the same fields.

If one environment shows stale setup cards while another shows suppression, mark as P0.

===== PART H — ADD OWNER PARITY
DIAGNOSTIC PAGE OR SECTION =====

Add an owner-only diagnostics section if feasible:

`/infra-health` or `/fno-diagnostics` or `/audit`

Section title:

`Signal / Telegram Parity`

Show:

- latest 10 notification_delivery_log records
- latest 10 canonical event parity checks
- duplicate blocked count
- test/dev blocked count
- last TEST_SYMBOL_BLOCKED event
- last DEV_ENV_BLOCKED event
- last Swing entry event parity status
- last F&O entry event parity status
- last exit event parity status
- real Telegram send enabled/disabled
- broker execution disabled status

This is diagnostic only.

No trade action buttons.

===== PART I — TESTS REQUIRED
=====

Add tests for:

1. UI projection uses canonical event values.
2. Telegram preview uses canonical event values.
3. Comparator catches entry mismatch.
4. Comparator catches SL mismatch.
5. Comparator catches target mismatch.
6. Comparator catches source mismatch.
7. Comparator catches status mismatch.
8. Comparator catches ID mismatch.

9. Replay existing event does not send Telegram.
10. Dry-run valid Swing event sends no Telegram.
11. Dry-run valid F&O event sends no Telegram.
12. TESTSTK dry-run blocks.
13. Yahoo dry-run blocks.
14. Stale dry-run blocks.
15. Dev dry-run blocks production Telegram.
16. Duplicate dry-run blocks second delivery.
17. F&O suppressed event cannot produce trade Telegram preview.
18. Exit event compares after close snapshot.
19. Fixture tests cannot hit real Telegram.
20. Production test endpoint rejects real trade-channel send.

Run:

```
pnpm --filter @workspace/api-server run typecheck
pnpm --filter @workspace/api-server run typecheck:libs
pnpm --filter @workspace/api-server exec vitest run src/lib/*trade*.test.ts
src/lib/*parity*.test.ts src/lib/*notification*.test.ts src/lib/
*telegram*.test.ts src/lib/*swing*.test.ts src/lib/*fno*.test.ts src/lib/
*paper*.test.ts src/routes/**/*test.ts
pnpm --filter @workspace/scanner exec vitest run
pnpm --filter @workspace/scripts run index:llm
pnpm --filter @workspace/scripts run index:llm:check
```

Use repo-correct paths if names differ.

===== PART J — REPORT REQUIRED
=====

Update:

`SIGNAL_DATA_NOTIFICATION_PARITY_FIX_REPORT.md`

Add section:

`Deterministic Parity Verification Harness`

Report:

1. What was added.
2. Endpoints added.
3. Safety gates.
4. Dry-run result.
5. Replay result.
6. TESTSTK result.
7. Swing parity result.
8. F&O parity result.
9. Field mismatch table.

10. Production-safe verification evidence.
11. Tests and counts.
12. LLM index result.
13. Remaining live-market smoke-check item, if any.

Final verdict must be exactly one:

- `SIGNAL_DATA_NOTIFICATION_PARITY_PROD_VERIFIED`
- `SIGNAL_DATA_NOTIFICATION_PARITY_DETERMINISTIC_PROD_VERIFIED_LIVE_SMOKE_PENDING`
- `SIGNAL_DATA_NOTIFICATION_PARITY_PARTIAL_SOURCE_GAP_REMAINS`
- `SIGNAL_DATA_NOTIFICATION_PARITY_BUILD_NOT_DEPLOYED`
- `ROLLBACK_REQUIRED`

===== ACCEPTANCE CRITERIA
=====

This task is complete only if:

- deterministic dry-run parity works now
- replay of existing production records works now
- no dry-run sends real Telegram
- no fixture sends real Telegram
- TESTSTK is blocked
- stale/Yahoo/dev events are blocked
- UI projection, Telegram preview, DB snapshot, paper snapshot, and notification log can be compared
- field mismatches are reported clearly
- F&O live/prod mismatch has diagnostic fields to explain differences
- owner can see parity health in diagnostics
- broker execution remains disabled
- no real orders are placed
- tests pass
- LLM index fresh
- production-safe verification completed

===== IMPORTANT FINAL NOTE
=====

Do not use this harness to pretend fake data is live.

The final real-market smoke check can still happen during market hours, but after this task we should no longer be blocked from verifying canonical field parity outside live session.